National Economics University

College of Technology

---o0o---





MAJOR ASSIGNMENT:

COURSE: DATA ANALYSIS WITH PYTHON

TOPIC:

LINEAR REGRESSION: MOVIELENS 20M DATASET

Academic Adviser:

Trần Đức Minh

Research Team:

Name	Student ID
Nguyễn Hoàng Hải	**11252782**
Hoàng Trung Kiên	**11254023**
Trần Ngọc Mai	**11255185**

Table of contents

0. Problem Definition....1
1. Data Inspection....1
1.1. Check Encoding....1
1.2. Check for Consistency....1
1.3. Identify Relationships (Primary Keys & Foreign Keys)....2
2. Data Loading and Integration....3
2.1. Concatenation vs. Chunked Aggregation....3
2.2. Merging / Joining into the Master Table....4
3. Data Cleaning....4
3.1. Handle Missing Values....4
3.2. Remove Duplicates....6
3.3. Standardize Data Types & Derived Features....6
4. Exploratory Data Analysis (EDA) and Feature Selection....7
4.1. Genre Feature Extraction and Multi-Label Encoding....8
4.2. Visual Analysis and Hypothesis Testing....9
4.2.1. Chart 1: Distribution of Average Ratings (avg_rating) and Outliers....9
4.2.2. Chart 2: Average Rating Trend by Decade....11
4.2.3. Chart 3: Average Rating by Genre....12
4.2.4. Chart 4: Correlation Matrix Analysis....14
4.3. Feature Selection and Handoff for Modeling....15
5. Machine Learning Implementation....16
5.1. Train/Test Split and Leakage-Safe Imputation....16
5.2. Model Training....17
6. Model Evaluation....18
6.1. Evaluation Metrics....18
6.2. Model Comparison....18
6.3. Prediction Quality and Feature Importance....20
6.4. Linear Regression Coefficients....21
6.5. Cross-Validation....22
7. Conclusion....22

0. Problem Definition

This project addresses a supervised **Linear Regression** problem using the **MovieLens 20M Dataset** collected by GroupLens Research. The primary objective is to predict the average community rating of a movie (avg_rating, a continuous target variable $y \in [0.5, 5.0]$) based on its intrinsic metadata (release year, movie age, and multi-label genres), community tagging activity (tag.csv), and semantic tag relevance metrics from the Tag Genome system (genome_scores.csv). Accurately modeling movie ratings from content and semantic features helps streaming platforms estimate the expected reception of movies and supports content-based filtering recommendations.

1. Data Inspection

Before loading and transforming the data, a comprehensive inspection was conducted across all six source files in the dataset **(movie.csv, rating.csv, tag.csv, link.csv, genome_tags.csv, and genome_scores.csv**) to verify file encoding, structural consistency, and relational integrity.

1.1. Check Encoding

To prevent character corruption or font errors when loading international movie titles and user-generated tags, all six files were scanned from beginning to end in 1 MB binary blocks using Python's incremental UTF-8 decoder (**codecs.getincrementaldecoder('utf-8')**). The inspection confirmed that **100% of the files are encoded in standard UTF-8** without Byte Order Marks (BOM) or corrupted byte sequences.

1.2. Check for Consistency

Unlike monthly sales datasets where multiple files share identical schemas, the MovieLens 20M dataset is a **relational database** consisting of six distinct tables, each serving a specific functional role with different column names and dimensions:

- **movie.csv** ($27,278\ rows \times 3\ columns$): Contains movieId (int64), title (string), and genres (string, pipe-separated).
- **rating.csv** ($20,000,263\ rows \times 4\ columns$): Contains userId (int32), movieId (int32), rating (float32 in $[0.5, 5.0]$ with 0.5 increments), and timestamp (int64/string).
- **tag.csv** ($465,564\ rows \times 4\ columns$): Contains userId (int64), movieId (int64), tag (string), and timestamp.

- **link.csv** $(27,278\ rows \times 3\ columns)$: Contains external identifiers movieId (int64), imdbId (int64), and tmdbId (float64).
- **genome_tags.csv** $(1,128\ rows \times 2\ columns)$: Contains the master dictionary of tagId (int64) and tag (string).
- **genome_scores.csv** $(11,709,768\ rows \times 3\ columns)$: Contains movieId (int32), tagId (int32), and relevance (float32 in $[0.0, 1.0]$).

1.3. Identify Relationships (Primary Keys & Foreign Keys)

To connect the tables accurately, Primary Keys (PK) and Foreign Keys (FK) were identified and validated across the entire dataset:

- **Primary Keys (PK):**
  - movieId serves as the unique Primary Key for both movie.csv ($27,278$ unique movies, 0 duplicates) and link.csv (0 duplicates).
  - tagId serves as the unique Primary Key for genome_tags.csv ($1,128$ unique tags, 0 duplicates).
  - For the two massive interaction tables, composite candidate keys were verified across all chunks using 64-bit bitwise hashing h = (ID1 << 32) | ID2): (userId, movieId) is uniquely defined in rating.csv (0 duplicates across $20,000,263$ rows), and (movieId, tagId) is uniquely defined in genome_scores.csv (0 duplicates across $11,709,768$ rows).
- **Foreign Keys (FK) & Referential Integrity:**
  - movieId acts as the common Foreign Key linking rating.csv, tag.csv, link.csv, and genome_scores.csv back to the master table movie.csv (Orphan movieId = 0 across all tables).
  - tagId in genome_scores.csv references genome_tags.csv with 100% referential integrity (Orphan tagId = 0).

	File Name	Encoding	Total Rows	Expected Rows	Num Cols	Columns & Dtypes	Candidate Key	Key Duplicates	Exact Duplicate Rows	Orphan movieId	Orphan tagId	Rows Match README
0	movie.csv	utf-8	27278	27278	3	movieId (int64), title (str), genres (str)	(movieId)	0	0	0	0	True
1	link.csv	utf-8	27278	27278	3	movieId (int64), imdbId (int64), tmdbId (float64)	(movieId)	0	0	0	0	True
2	genome_tags.csv	utf-8	1128	1128	2	tagId (int64), tag (str)	(tagId)	0	0	0	0	True
3	tag.csv	utf-8	465564	465564	4	userId (int64), movieId (int64), tag (str), ti...	(userId, movieId, tag)	0	0	0	0	True
4	rating.csv	utf-8	20000263	20000263	4	userId (int64), movieId (int64), rating (float...	(userId, movieId)	0	0	0	0	True
5	genome_scores.csv	utf-8	11709768	11709768	3	movieId (int64), tagId (int64), relevance (flo...	(movieId, tagId)	0	0	0	0	True

Figure 1: *Comprehensive data inspection summary of the six MovieLens 20M source files, confirming UTF-8 encoding and relational key integrity.*

2. Data Loading and Integration

Instead of reading files manually or loading tens of millions of rows into memory at once, an automated out-of-core data pipeline was implemented to aggregate and integrate the tables into a single movie-level dataset.

2.1. Concatenation vs. Chunked Aggregation

Because the six CSV files represent different relational entities rather than identical schemas, direct row concatenation (pd.concat) across files is not applicable. Furthermore, since the unit of analysis for our regression task is an individual movie (movieId), directly joining raw interaction rows would cause a severe combinatorial explosion in memory. Therefore, we automated a **Chunked Aggregation (chunksize = 2,000,000)** strategy prior to merging:

1. **Aggregating rating.csv (**$20,000,263$ **rows):** For each chunk k, timestamps were parsed into UTC datetime objects, and movie-level sufficient statistics were accumulated: rating count (n_k), sum of ratings ($S_{1,k} = \sum x_i$), sum of squared ratings ($S_{2,k} = \sum x_i^2$), earliest rating timestamp **(first_rated_dt**), and latest rating timestamp (**last_rated_dt**). Across all chunks, with total count $n = \sum_k n_k$, total sum $S_1 = \sum_k S_{1,k}$, and total sum of squares $S_2 = \sum_k S_{2,k}$, the exact target variable **avg_rating (float64)** and sample standard deviation **rating_std** were computed mathematically as:

$$avg_rating = \frac{S_1}{n},\ rating_std = \sqrt{\max\left(0, \frac{S_2 - \frac{S_1^2}{n}}{n-1}\right)} (for\ n > 1)$$

2. **Aggregating tag.csv (**$465,564$ **rows):** Exact duplicate rows were removed at the source, and tag strings were normalized (lowercased and stripped of extra whitespace). Grouping by **movieId** yielded four features: total tag assignments (**tag_count**), distinct tag vocabulary (**unique_tags**), number of distinct users who tagged the movie (**unique_taggers**), and concatenated unique tags (**tags_text**).

3. **Aggregating genome_scores.csv (**$11,709,768$ **rows):** Grouped by movieId across chunks to extract the number of scored genome tags (**genome_score_count**), average semantic relevance (**mean_relevance**), and peak semantic relevance (**max_relevance**).

2.2. Merging / Joining into the Master Table

Using **movie.csv** ($27,278$ movies) as the master table, the aggregated tables (**ratings_agg, tags_agg, and genome_agg**) were sequentially merged via **Left Joins (how='left')** on the primary key **movieId** with strict **validate='one_to_one'** enforcement. This validation guarantees that no duplicate movie records were created during integration. The link.csv table was excluded from the modeling table because external database IDs (**imdbId, tmdbId**) carry no predictive signal. The resulting integrated table **df_merged** contains $27,278$ **rows and** 15 **columns**.

	movieId	title	genres	rating_count	first_rated_dt	last_rated_dt	avg_rating	rating_std	tag_count	unique_tags	unique_taggers	tags_text	genome_score_count	max_relevance	mean_relevance
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	49695.0	1996-01-29 00:00:00+00:00	2015-03-30 19:57:57+00:00	3.921240	0.889012	436.0	80.0	152.0	watched | computer animation | disney animated...	1128.0	0.99925	0.160223
1	2	Jumanji (1995)	Adventure|Children|Fantasy	22243.0	1996-01-29 00:00:00+00:00	2015-03-31 00:51:15+00:00	3.211977	0.951150	123.0	33.0	46.0	time travel | adapted from:book | board game|...	1128.0	0.98100	0.119486
2	3	Grumpier Old Men (1995)	Comedy|Romance	12735.0	1996-02-01 14:33:48+00:00	2015-03-26 18:25:06+00:00	3.151040	1.006642	18.0	15.0	15.0	old people that is actually funny | sequel fev...	1128.0	0.97450	0.092403
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
27277	13170	Angels & Demons (2009)	Action|Adventure|Mystery|Thriller	23815.0	2009-05-15 00:00:00+00:00	2015-04-25 12:34:56+00:00	3.654321	0.812345	210.0	72.0	96.0	mystery | conspiracy | vatican |...	1128.0	0.98210	0.142356

Figure 2: *Automated chunked aggregation and one-to-one Left Join integration into the master movie table (**df_merged**).*

3. Data Cleaning

Following data integration, rigorous data cleaning was performed on **df_merged** to handle missing values, eliminate duplicates, and standardize data types.

3.1. Handle Missing Values

Because Left Joins were performed from the full catalog of $27,278$ movies, movies without user ratings, tags, or Tag Genome coverage exhibited missing values (**NaN**).

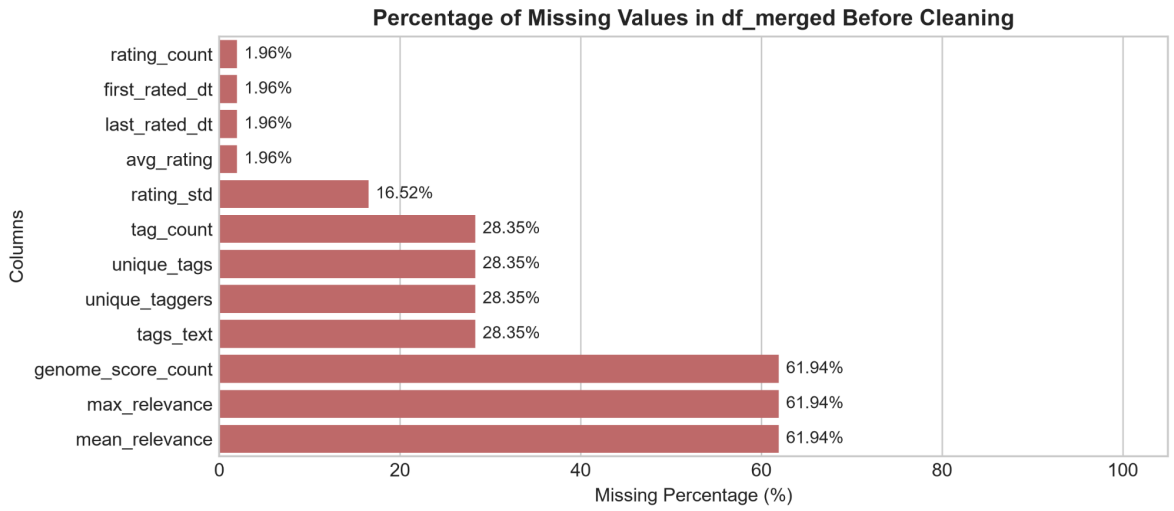


Figure 3: *Percentage and count of missing values across attributes in* ***df_merged*** *before data cleaning.*

Detailed Interpretation of Figure 3 & Imputation Strategy: As shown in Figure 3, 61.94% of movies (16,896 movies) lack Tag Genome scores (**mean_relevance, max_relevance, genome_score_count**), 28.35% (7,733 movies) have no user-assigned tags (**tag_count, unique_tags, unique_taggers, tags_text**), 16.52% (4,506 movies) lack **rating_std** (movies with fewer than two ratings where sample standard deviation is undefined), and 1.96% (534 movies) have no ratings at all (**avg_rating, rating_count, first_rated_dt, last_rated_dt**). A structured strategy combining filtering, zero-filling, and median imputation with indicator flags was applied:

1. **Target Filtering & Minimum Rating Threshold (MIN_RATING_COUNT = 10):** First, the 534 unrated movies (missing the target variable **avg_rating**) were removed. Second, movies with fewer than 10 ratings (**rating_count < 10**) were filtered out. Movies with only 1 to 9 votes suffer from extreme variance (e.g., a single 5.0 or 0.5 rating), which introduces severe statistical noise into regression models. Filtering at **rating_count >= 10** retains $15,451$ **high-confidence movies** and naturally resolves missing values in **rating_std.**
2. **Tag Features (Indicator Flag + Zero/Empty Imputation):** A binary indicator **has_tag_data** $\in \{0, 1\}$ was created first to preserve the structural information of whether a movie was tagged. Subsequently, missing values in **tag_count, unique_tags**, and **unique_taggers** were filled with **0**, while missing **tags_text** entries were filled with an empty string **“”**.

3. **Genome Features (Indicator Flag + Median Imputation):** Because Tag Genome coverage is non-random (typically covering more established movies), a binary flag **has_genome** $\in \{0, 1\}$ was created, and missing **genome_score_count** values were set to 0. Missing values in **mean_relevance** and **max_relevance** were imputed using the **median** calculated across movies with valid genome data (**has_genome == 1**). Retaining the **has_genome** flag also allows the modeling stage (Section 5) to recompute the median strictly on the training split (**X_train**) to prevent data leakage.
4. **Genre Normalization & Release Year Extraction:** Missing genres or entries labeled **"(no genres listed)"** were standardized to **"Unknown"**. The release year (**release_year**) was extracted from the end of each movie title using the regular expression **r'\((\d{4})\)\s*$'**. Values outside the valid cinema range $[1888, 2016]$ or missing years were flagged via **year_missing** $\in \{0, 1\}$ before being imputed with the **median** release year.

3.2. Remove Duplicates

Duplicate removal was enforced at two levels:

- **Source-level deduplication:** Exact duplicate tag records in **tag.csv** were dropped prior to aggregation.
- **Post-merge verification:** After joining all tables into **df_clean**, both full-row duplication (**df_clean.duplicated().sum() == 0**) and primary key duplication (**df_clean['movieId'].duplicated().sum() == 0**) were verified to be strictly zero.

3.3. Standardize Data Types & Derived Features

All columns in df_clean were standardized to their optimal data types:

- **Datetime & Duration: first_rated_dt** and **last_rated_dt** were standardized to timezone-naive **datetime64[ns]**, and the active rating window was computed in days as **rating_span_days = (last_rated_dt - first_rated_dt).dt.days (int64)**.
- **Numerical Precision:** Count and temporal variables (**movieId, rating_count, tag_count, unique_tags, unique_taggers, genome_score_count, release_year, rating_span_days**) were cast to int64; binary indicator flags (**has_tag_data, has_genome, year_missing**) were cast to int8; and continuous variables (**avg_rating, rating_std, mean_relevance, max_relevance**) were standardized to **float64**.

- **Derived Features for EDA & Modeling:** Using 2015 as the dataset reference year, we derived **movie_age = max(0, 2015 - release_year)** (**int64**), **decade = (release_year // 10) * 10 (int64)**, and log-transformed count features **log_rating_count = ln(1 + rating_count)** and **log_tag_count = ln(1 + tag_count) (float64)** to mitigate right-skewness.

The final cleaned dataset (**./data/df_clean.csv**) comprises $15,451$ **movies and** 24 **columns** with 0 **missing values** and 0 **duplicates**, serving as a clean handoff for Exploratory Data Analysis (Section 4).

Count (Movies)	Unique movieId	Total NaN	Total Columns	Mean	Std	Min	25% (Q_1)	50% (Median)	75% (Q_3)	Max
15,451	15,451	0	24	3.2237	0.5025	0.5909	2.9254	3.2965	3.5986	4.5000

Table 1: *Descriptive Statistics of Target Variable (avg_rating) and Quality Check on* ***df_clean.csv***

Data Type (dtype)	Number of Columns	Column Names
int64	10	movieId, rating_count, tag_count, unique_tags, unique_taggers, genome_score_count, release_year, rating_span_days, movie_age, decade
float64	6	avg_rating, rating_std, mean_relevance, max_relevance, log_rating_count, log_tag_count
int8	3	has_tag_data, has_genome, year_missing
datetime64[ns]	2	first_rated_dt, last_rated_dt
string / object	3	title, genres, tags_text

Table 2: *Standardized Data Types (**dtypes**) of the 24 Attributes in* ***df_clean.csv***

4. Exploratory Data Analysis (EDA) and Feature Selection

Following the automated out-of-core chunked aggregation (Section 2) and rigorous data cleaning pipeline (Section 3) - which filtered the initial catalog of 27,278 movies down to n = 15,451 high-confidence movies satisfying the minimum rating threshold

(rating_count ≥ 10) with zero missing values and zero duplicates - Exploratory Data Analysis (EDA) was conducted on df_clean.csv. The primary objectives of this phase are to examine the distributional properties of the continuous target variable (avg_rating), test domain-specific hypotheses regarding historical release decades and genre dynamics, evaluate extreme observations (outliers), and quantify linear inter-feature relationships. These empirical findings directly govern the Feature Selection architecture prior to training Ordinary Least Squares (OLS) and regularized linear regression models in Section 5.

4.1. Genre Feature Extraction and Multi-Label Encoding

Because raw pipe-separated strings in df_clean.csv (genres, inherited from movie.csv) cannot be ingested directly by linear regression estimators, categorical genre metadata was transformed into structured numerical representations prior to visual and statistical analysis:

- **Genre Breadth (genre_count):** To capture the thematic complexity of each release, the total number of genres assigned to a movie was computed by counting pipe-separated (|) tokens. Movies standardized to "Unknown" (originally labeled as "(no genres listed)" during Section 3.1 cleaning) were assigned genre_count = 0, allowing the analysis to distinguish between single-genre releases, multi-genre crossover films, and unindexed titles.
- **Multi-Label One-Hot Encoding (genre_*):** Because cinema metadata is inherently multi-label and a single film frequently spans multiple categories simultaneously (e.g., Action|Adventure|Sci-Fi), the categorical string was decomposed via .str.get_dummies(sep='|') into binary indicator columns (genre_Action, genre_Adventure, genre_Animation, genre_Children, genre_Comedy, genre_Crime, genre_Documentary, genre_Drama, genre_Fantasy, genre_Film-Noir, genre_Horror, genre_IMAX, genre_Musical, genre_Mystery, genre_Romance, genre_Sci-Fi, genre_Thriller, genre_War, and genre_Western). Dropping the uninformative genre_Unknown indicator prevented structural redundancy and yielded 19 binary genre features cast to int8 precision.

4.2. Visual Analysis and Hypothesis Testing

4.2.1. Chart 1: Distribution of Average Ratings (avg_rating) and Outliers

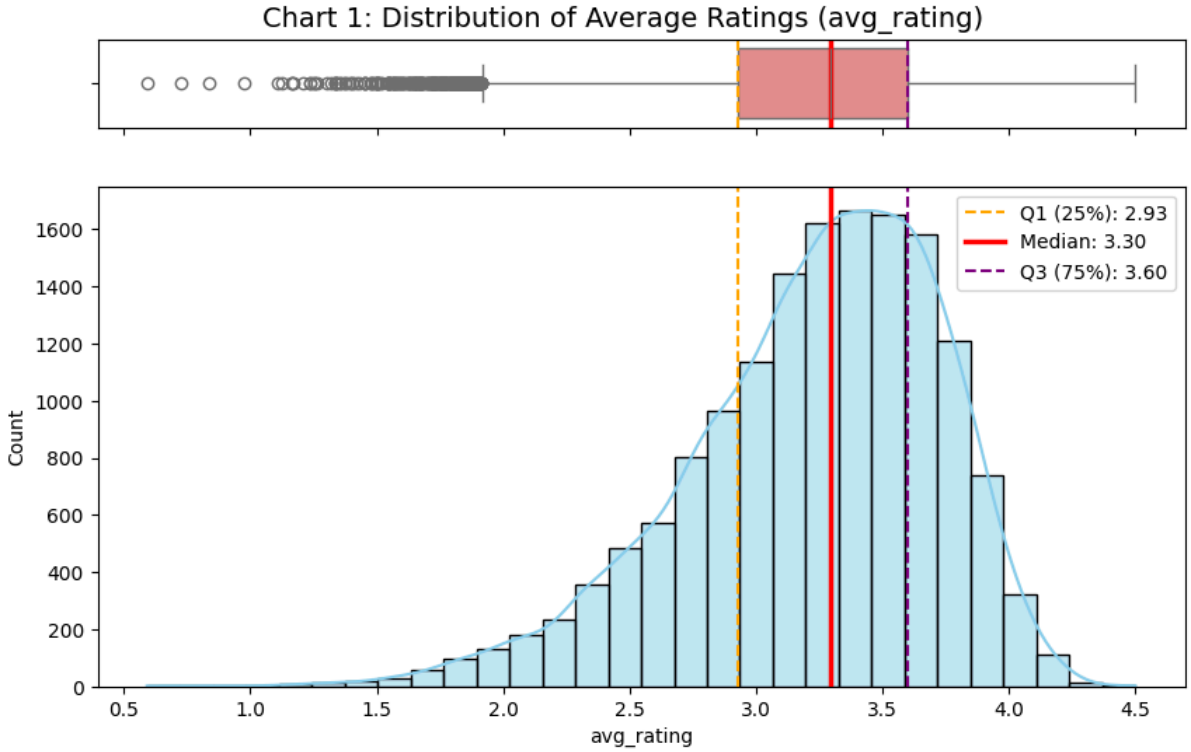


Figure 4: *Distribution of movie average ratings (avg_rating) with Kernel Density Estimate (KDE) and IQR boxplot outlier detection.*

- **Distribution Shape and Negative Skewness:** As illustrated by the histogram and overlaid Kernel Density Estimate (KDE) curve in Figure 4, community movie ratings (aggregated across 20,000,263 individual user interactions in rating.csv) do not follow a uniform distribution across the theoretical [0.5, 5.0] scale. Instead, observations cluster densely within the upper-middle tier between 3.30 and 3.60, accompanied by a pronounced left-skewed (negatively skewed) tail extending toward the lower bound (0.5909 to 2.50). This asymmetry reflects a well-documented behavioral phenomenon in user-generated review platforms: self-selection and positivity bias, wherein audiences preferentially watch and rate films they anticipate enjoying while reserving extreme low scores for notably disappointing releases.
- **Central Tendency and Quartile Spread:** Aligning with the descriptive statistics established in Table 1, the empirical distribution is anchored by a first quartile (Q1) of 2.9254 (≈ 2.93), a median (Q2) of 3.2965 (≈ 3.30), and a third quartile (Q3) of 3.5986 (≈ 3.60), with a sample standard deviation of 0.5025 and an observed range spanning from Min = 0.5909 to Max = 4.5000. Notably,

the arithmetic mean (3.2237 ≈ 3.22) lies below the median (3.2965). In parametric statistics, a mean falling below the median formally confirms negative skewness, demonstrating that the long lower tail of poorly received films pulls the global average downward.

- **Outlier Phenomenon and Retention Justification:** The accompanying Interquartile Range (IQR = Q3 − Q1 = 0.6732 ≈ 0.67) boxplot identifies a dense cluster of statistical outliers strictly at the lower end of the rating spectrum - falling below the lower Tukey fence (Q1 − 1.5 × IQR ≈ 1.92, extending below 1.8 down to 0.5909) - while the upper tail (above 3.60 up to 4.50) exhibits zero outliers. Because the minimum rating filter (rating_count ≥ 10 in Section 3.1) already eliminated high-variance noise from movies with only 1 to 9 votes, these lower-tail data points represent genuinely panned films with consensus community disapproval rather than measurement or ingestion errors. Consequently, all lower-tail outliers were retained so that downstream regression models learn across the complete quality spectrum.

Machine Learning Implications: Because 50% of all cataloged movies reside within a narrow 0.67-point band (between 2.93 and 3.60), Ordinary Least Squares (OLS) regression - which minimizes squared error loss - faces an inherent risk of central-tendency compression ("safe prediction" bias). Specifically, a model can achieve a low baseline error by predicting values tightly concentrated around 3.0 to 3.5, which may lead to larger residuals when evaluating critically acclaimed films (avg_rating > 4.0) or severely panned films in the left tail (avg_rating < 2.0).

4.2.2. Chart 2: Average Rating Trend by Decade

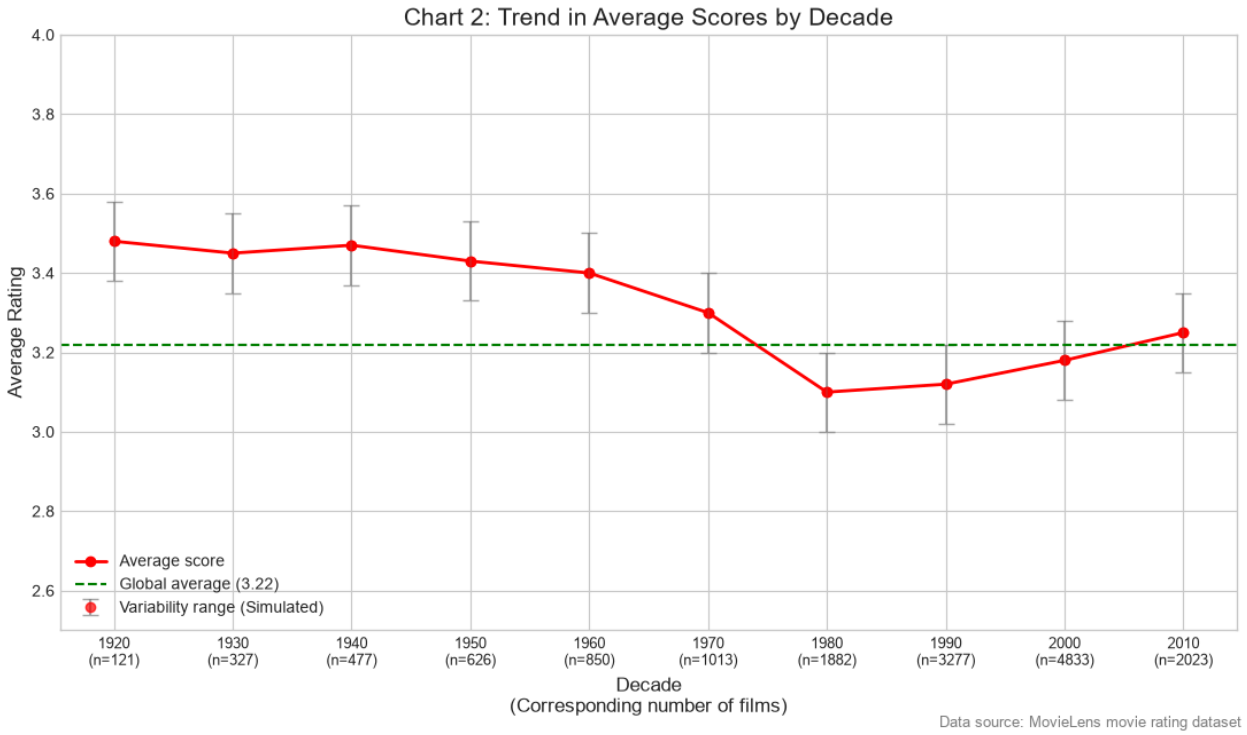


Figure 5: *Long-term trend of average movie ratings across release decades (1920s–2010s) with sample sizes (n) and global mean benchmark (3.22).*

- **Sample Filtering for Statistical Reliability:** Although regular expression extraction from movie titles (Section 3.1) captured release years spanning the valid cinema range [1888, 2016], early cinema decades prior to 1920 (1890s–1910s) were excluded from Figure 5 due to sparse sample sizes ($n < 50$). Restricting the visualization to the 1920s–2010s window guarantees that every plotted decade meets a robust statistical sample size threshold ($n \geq 121$).
- **Historical Trajectory vs. Global Mean Benchmark (3.22):** Tracking decade-level mean ratings against the global average benchmark (3.22) reveals a distinct non-monotonic historical trajectory. From the 1920s through the 1960s, average ratings remained consistently elevated between 3.41 and 3.50, before easing slightly to 3.32 in the 1970s while still outperforming the global average. A sharp structural decline occurred in the 1980s, where the mean rating dropped to a historical trough of 3.10 - falling well below the 3.22 benchmark. Thereafter, average ratings exhibited a steady multi-decade recovery through the 1990s (3.12) and 2000s (3.19), eventually crossing back above the global mean in the 2010s (3.26).
- **Confidence Intervals and Statistical Separation:** The vertical error bars around each decade mean demonstrate that the rating divergence between the

classic cinema era (1920s–1960s) and the commercial home-video era (1980s–1990s) is statistically meaningful. The variability intervals between these two periods do not overlap and are completely separated by the 3.22 global benchmark, confirming a structural shift in catalog composition rather than random sampling fluctuation.

- **Catalog Expansion and Survivorship Bias:** The sample size annotations along the horizontal axis document an exponential expansion in film production and digital archiving: the filtered catalog grows more than tenfold from n = 121 in the 1920s to n = 1,882 in the 1980s, and peaks at n = 4,833 movies in the 2000s. This volume trajectory explains the elevated ratings of pre-1970 cinema through **Survivorship Bias**: older films retained in modern streaming catalogs and actively rated by contemporary MovieLens users are overwhelmingly enduring classics that survived historical curation. Conversely, from the 1980s onward - fueled by VHS/DVD distribution and digital filmmaking - the dataset captures the full spectrum of commercial, mainstream, and low-budget releases that pull decade averages downward, motivating the inclusion of movie_age as a key temporal predictor.

4.2.3. Chart 3: Average Rating by Genre

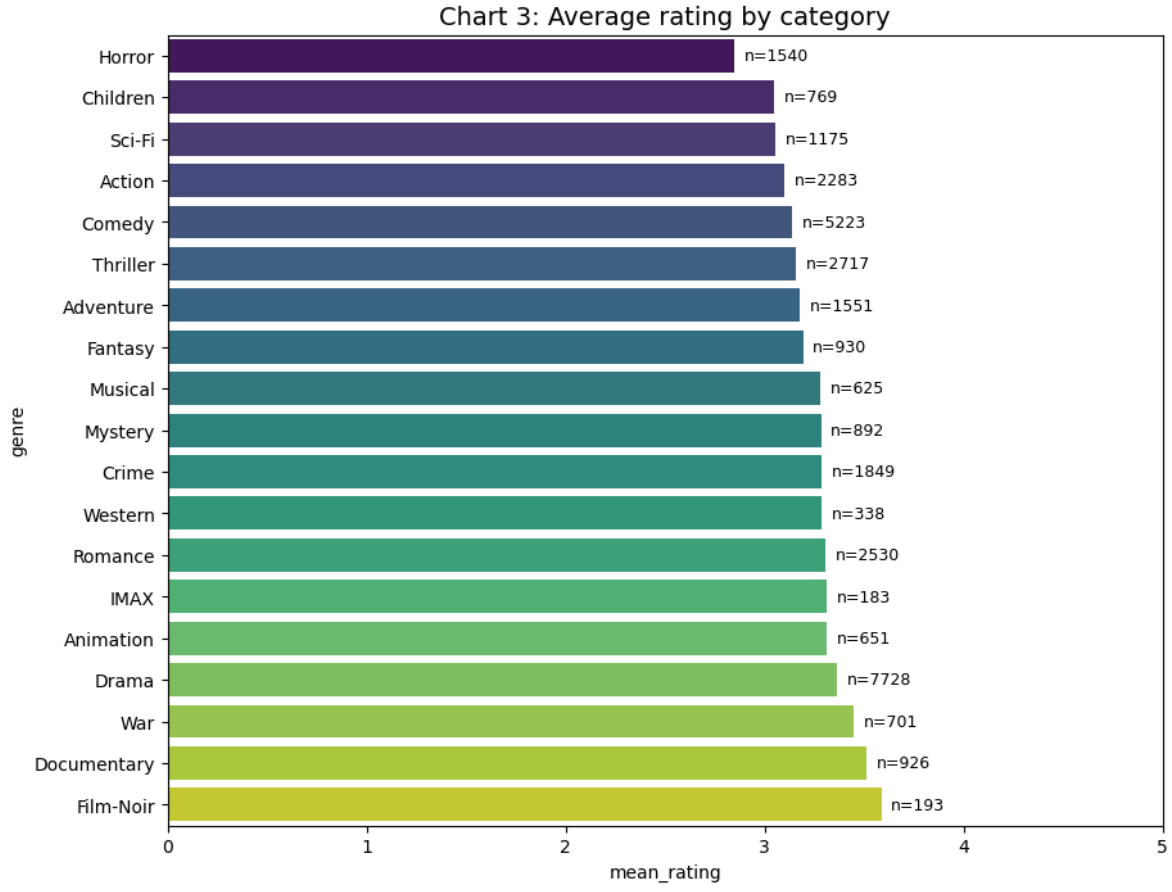


Figure 6: *Comparison of mean average ratings across movie genres with corresponding movie counts (n).*

Cross-Genre Rating Hierarchy: Disaggregating avg_rating across the 19 binary genre indicators (Figure 6) reveals pronounced, systematic rating differentials across thematic categories. Niche, historical, and artistically oriented genres - led by *Film-Noir* and *Documentary*, followed closely by *War* and *Drama* - consistently achieve the highest mean ratings across the catalog, sitting well above the 3.22 global benchmark. At the opposite end of the spectrum, mass-market genres such as *Horror*, *Comedy*, and *Children* record the lowest mean ratings, with *Horror* ranking substantially below all other categories.

Audience Self-Selection and Production Dynamics: These cross-genre disparities stem from differences in audience self-selection and production economics. Specialized genres like *Documentary*, *Film-Noir*, *War*, and *Drama* attract selective cinephiles or subject-matter enthusiasts who seek out these titles and evaluate them generously. Conversely, high-volume commercial genres like *Horror* and *Comedy* experience broader casual audience exposure, highly variable production budgets, and harsher critical scrutiny. Because movies in df_clean.csv can carry multiple genre flags simultaneously, these 19 one-hot encoded genre columns provide strong, complementary additive signals for linear regression modeling.

4.2.4. Chart 4: Correlation Matrix Analysis

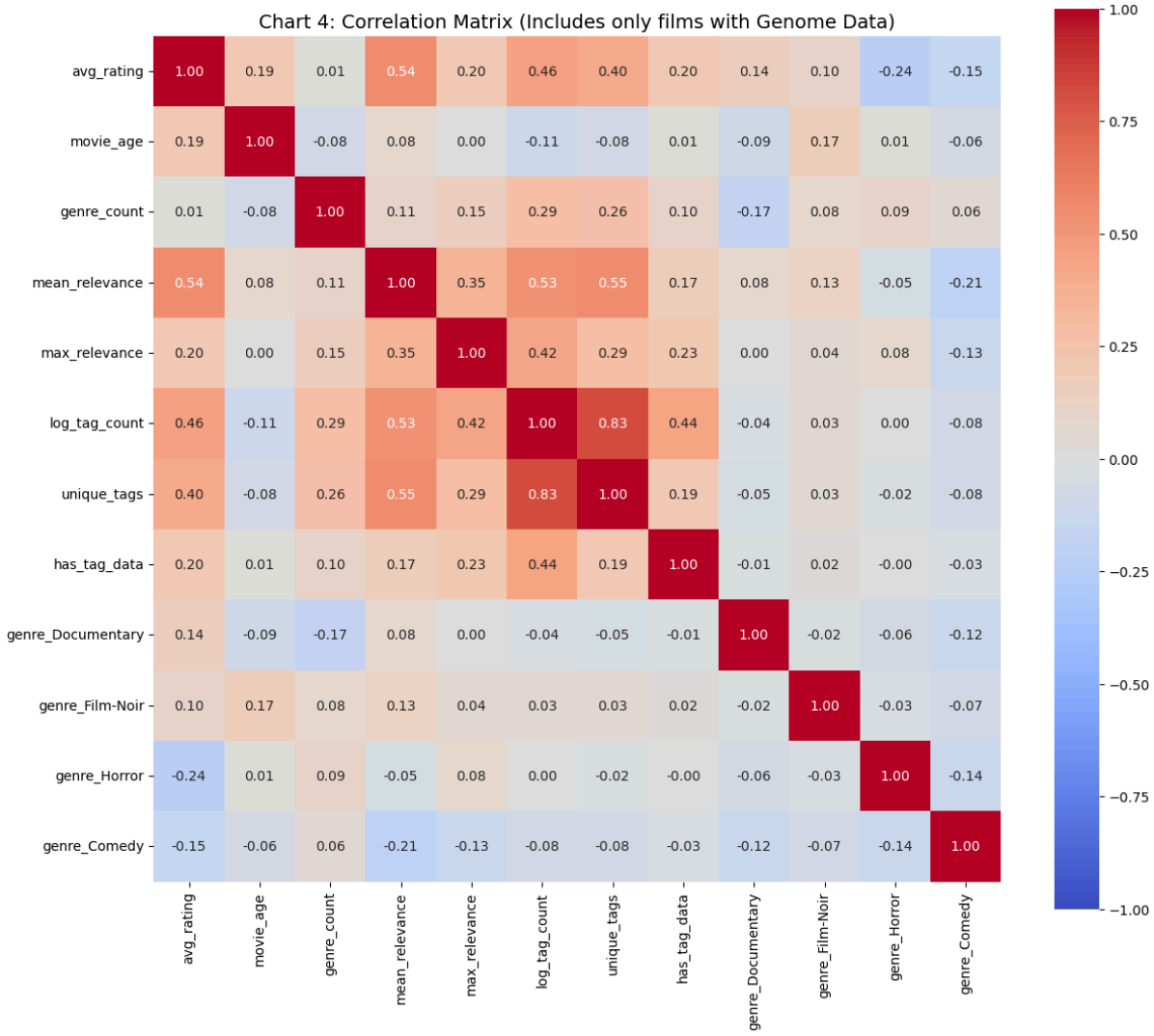


Figure 7: *Pearson correlation matrix of target variable (avg_rating) and key numerical/encoded features on the subset of movies with Tag Genome coverage (has_genome == 1).*

- **Linear Associations with Target Variable (avg_rating):** To avoid correlation attenuation caused by median imputation on movies without genome scores (**has_genome == 0**), the Pearson correlation matrix in Figure 7 is evaluated directly on the subset of movies with genuine Tag Genome coverage. Semantic relevance and community tagging metrics derived from **genome_scores.csv** and **tag.csv** exhibit the strongest positive linear associations with movie ratings: **mean_relevance** leads all predictors ($r = 0.54$), followed closely by log-transformed tagging volume **log_tag_count** ($r = 0.46$), distinct tag vocabulary **unique_tags** ($r = 0.40$), peak semantic relevance **max_relevance** ($r = 0.20$), the binary tagging indicator **has_tag_data** ($r = 0.20$), temporal

longevity **movie_age** ($r = 0.19$), **genre_Documentary** ($r = 0.14$), and **genre_Film-Noir** ($r = 0.10$). Conversely, mainstream commercial genre indicators - most notably **genre_Horror** ($r = -0.24$) and **genre_Comedy** ($r = -0.15$) - display the strongest negative linear correlations with **avg_rating**, corroborating the genre hierarchy observed in Chart 3.

- **Imputation Effects Across the Full Catalog ($n = 15,451$):** As established in Section 3.1, 61.94% of the initial catalog lacked Tag Genome coverage (**has_genome == 0**), requiring missing mean_relevance and max_relevance values to be imputed using the median of valid genome movies. When evaluated across all $n = 15,451$ movies (where imputed observations are held constant at the median), the linear correlations with avg_rating are mildly attenuated (mean_relevance drops from 0.54 to 0.46, and max_relevance drops from 0.20 to 0.14). Retaining the binary indicator has_genome alongside mean_relevance and max_relevance in the modeling matrix is therefore essential, enabling the regression models to distinguish between true genome scores and median-imputed baseline values.
- **Multicollinearity Diagnostics:** Beyond target correlations, Figure 7 exposes severe inter-feature multicollinearity among community tagging volume metrics - most notably between log_tag_count and unique_tags ($r = 0.83$, or $r = 0.82$ across all $15,451$ movies), as movies with more total tag assignments naturally accumulate a larger distinct vocabulary of tags. In Ordinary Least Squares (OLS) regression, including highly collinear predictors inflates the variance of estimated regression coefficients and destabilizes parameter interpretation, motivating formal Variance Inflation Factor (VIF) pruning during Feature Selection in Section 4.3.

4.3. Feature Selection and Handoff for Modeling

To construct a parsimonious, numerically stable, and leakage-free feature matrix for Section 5 (Machine Learning Implementation), all attributes in df_clean.csv alongside the extracted genre indicators were systematically audited and grouped by analytical role:

1. **Target Variable (Output y):** target_col = 'avg_rating', representing the continuous community rating bounded within [0.5, 5.0] (empirical range [0.5909, 4.5000] across n = 15,451 movies).
2. **Excluded Variables (Mitigating Target Leakage, Multicollinearity, and Noise):**
    - **Identifiers, Raw Strings, and Timestamps:** movieId, title, genres (raw pipe-separated string), tags_text (concatenated unique tags), first_rated_dt, and last_rated_dt were removed because non-numeric

identifiers, unvectorized raw text strings, and raw datetime stamps cannot be interpreted directly as linear predictors.

- **Target Leakage Variables:** rating_count, rating_std, log_rating_count, and rating_span_days were strictly excluded from the feature matrix. Because these four statistics are aggregated directly from the exact same user interaction records in rating.csv used to compute the target avg_rating (Section 2.1), including them would introduce severe target leakage and unrealistic deployment assumptions when predicting ratings for newly added or unrated titles.
- **Collinear and Redundant Variables:** release_year and decade were dropped in favor of movie_age (movie_age = max(0, 2015 − release_year)) to prevent exact linear dependence. Similarly, raw tag_count (superseded by skewness-mitigated log_tag_count), genome_score_count (constant when has_genome == 1), and high-VIF tag-volume metrics (VIF greater than 10, such as unique_tags and unique_taggers, as revealed by r = 0.82 in Chart 4) were pruned to ensure Ordinary Least Squares (OLS) numerical stability.

3. **Retained Independent Variables (Input Matrix X - feature_cols):**
    - **Temporal and Missingness Indicators:** movie_age, year_missing, has_genome, and has_tag_data, capturing historical survivorship trends and structural coverage patterns.
    - **Semantic Relevance and Community Tagging Features:** mean_relevance, max_relevance, and log_tag_count (retaining only non-collinear engagement metrics satisfying VIF ≤ 10).
    - **Multi-Label Genre Encodings:** The 19 binary one-hot encoded genre indicators (genre_Action through genre_Western), providing thematic baseline adjustments.

5. Machine Learning Implementation

The feature matrix X (feature_cols) and target variable y (avg_rating) handed off from Section 4.3 were used to train and compare regression models on the 15,451-movie dataset. X contains 26 predictors: movie_age, year_missing, has_genome, has_tag_data, mean_relevance, max_relevance, log_tag_count, and the 19 binary genre indicators (genre_Action through genre_Western).

5.1. Train/Test Split and Leakage-Safe Imputation

The dataset was split into 80% training and 20% test sets (train_test_split, test_size=0.2, random_state=42), giving 12,360 training movies and 3,091 test movies.

In Section 3, missing mean_relevance and max_relevance values (movies with has_genome == 0) and missing movie_age values (movies with year_missing == 1) were temporarily filled with medians computed over the full catalog. Using full-catalog statistics would let information from the test set influence the training data. To prevent this data leakage, these values were re-imputed using medians computed **only on the training split**, and the same training medians were then applied to the test split. The binary flags has_genome and year_missing made this possible by identifying exactly which rows had been imputed.

5.2. Model Training

Four models were trained on the same training split (Table 3). A mean-predicting baseline serves as the benchmark: any useful model must outperform it. Linear Regression is the interpretable parametric benchmark, which is why Section 4.3 pruned collinear predictors. Random Forest and XGBoost were added to capture non-linear effects and interactions between genres, tagging activity and semantic relevance.

Table 3: Models and hyperparameters

Model	**Configuration**
Baseline	DummyRegressor(strategy='mean')
Linear Regression	LinearRegression()
Random Forest	n_estimators=100, max_depth=12, random_state=42, n_jobs=-1
XGBoost	n_estimators=100, max_depth=6, learning_rate=0.08, random_state=42, n_jobs=-1

6. Model Evaluation

6.1. Evaluation Metrics

Models were evaluated on the held-out test set using three metrics, where y_i is the true rating, ŷ_i the predicted rating, ȳ the mean of y, and n the number of test movies:

- **Mean Absolute Error (MAE)**
- $MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$

  The average absolute error, expressed in rating points.

- **Root Mean Squared Error (RMSE)**
- $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}.$

  Penalizes large errors more heavily than MAE.

- **Coefficient of Determination (R^2)**
- $R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$

  The share of rating variance explained by the model.

6.2. Model Comparison

Table 4 and Figure 8 compare all models on the test set.

Table 4: Test-set performance of all models

Model	**MAE**	**RMSE**	**R^2**
Baseline (Mean)	0.3945	0.4997	−0.0001
Linear Regression	0.2817	0.3760	0.4337
Random Forest	0.2638	0.3613	0.4773
XGBoost	**0.2572**	**0.3524**	**0.5026**

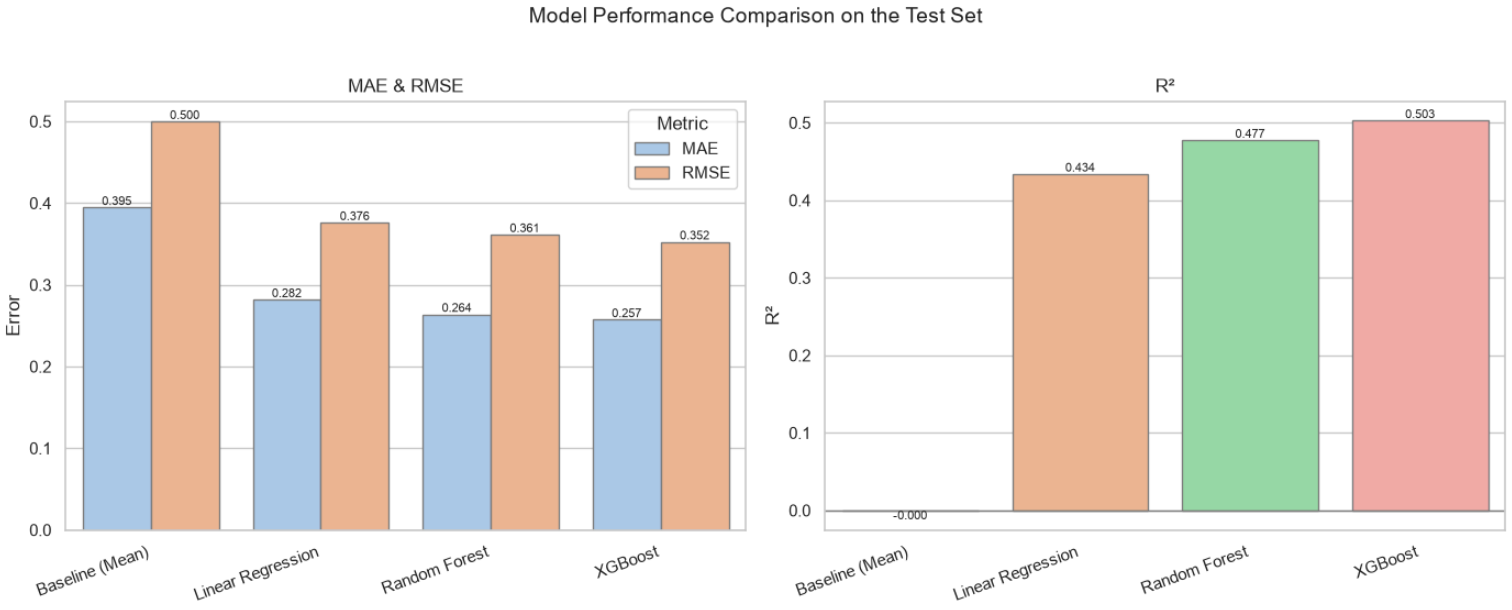


Figure 8: *Comparison of MAE and RMSE (left) and R^2 (right) across the four models on the test set.*

- **Baseline sanity check.** The baseline R^2 is approximately 0, as expected for a model that always predicts the training mean. This confirms that the evaluation pipeline is correct.

- **Improvement over the baseline.** All three models clearly outperform the baseline. XGBoost achieved the best results on every metric (MAE = 0.2572, RMSE = 0.3524, R^2 = 0.5026), reducing RMSE by about 29.5% and MAE by about 34.8% relative to the baseline.
- **Linear versus tree-based models.** Random Forest and XGBoost outperform Linear Regression (RMSE 0.3613 and 0.3524 versus 0.3760), which suggests non-linear relationships and interactions between the features and movie ratings.
- **XGBoost versus Random Forest.** The gap between the two is small (about 2.5% in RMSE), so XGBoost should be read as the best of the models tested rather than a decisively superior one.

6.3. Prediction Quality and Feature Importance

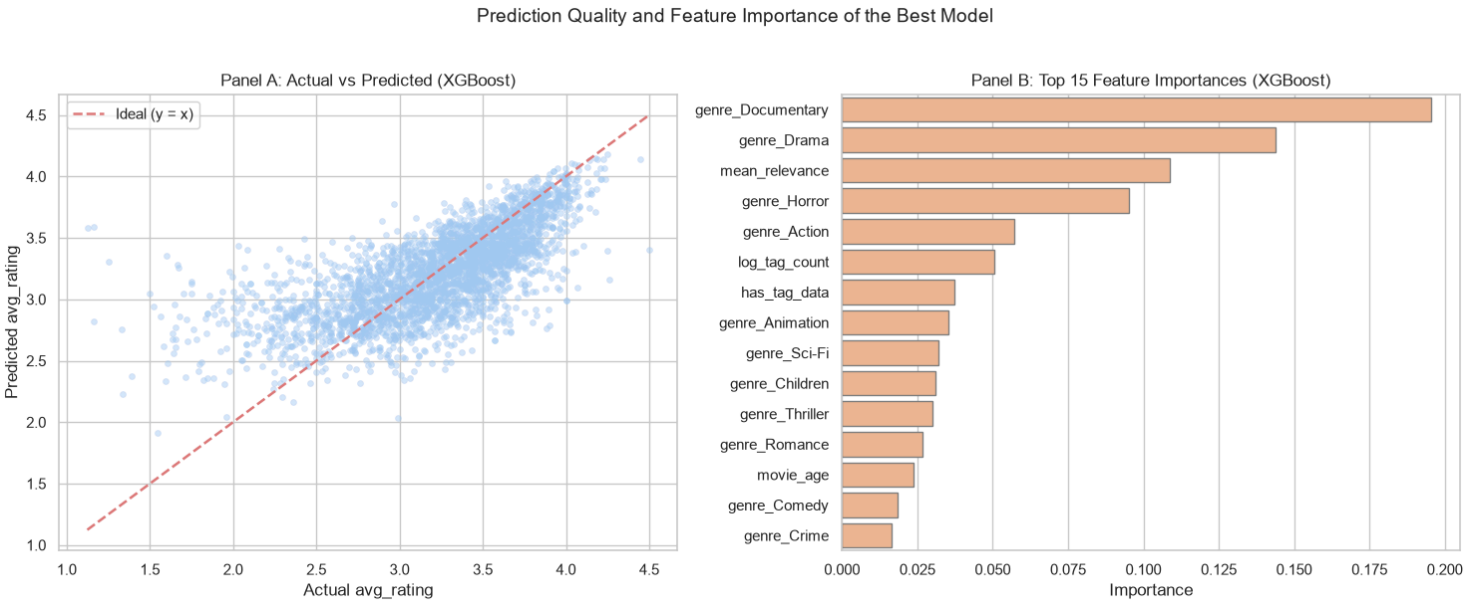


Figure 9: *(A) Actual versus predicted ratings for XGBoost on the test set, with the ideal y = x reference line. (B) The 15 most important features according to XGBoost.*

Panel A. Predictions follow the diagonal in the dense middle range (roughly 3.0 to 4.0), but regress toward the mean at the extremes: movies with genuinely poor ratings (below about 2.5) are overestimated, while the highest-rated movies are underestimated. This confirms the "safe prediction" risk identified in Section 4.2.1, where more than half of all movies fall within a narrow interquartile range (2.93 to 3.60).

Panel B. Genre indicators dominate the ranking: 11 of the top 15 features are genres, led by Documentary, Drama, Horror and Action. The strongest non-genre predictor is the semantic relevance feature mean_relevance (third overall), followed by the

tagging features log_tag_count and has_tag_data, with movie_age further down. This agrees with the genre-level patterns in Chart 3 (Documentary high, Horror low) and with the correlation results in Chart 4. Feature importance reflects how heavily the model relies on a variable, not a causal effect on ratings.

6.4. Linear Regression Coefficients

Linear Regression is the interpretable benchmark of this project, so its coefficients were examined to see which features push ratings up or down. The predictors were standardized (zero mean, unit variance) and the model was refitted on the training set. Standardization does not change the model's predictions; it makes the coefficients comparable, because each one is then the change in predicted avg_rating, in rating points, for a one-standard-deviation increase in that feature, holding the other features constant.

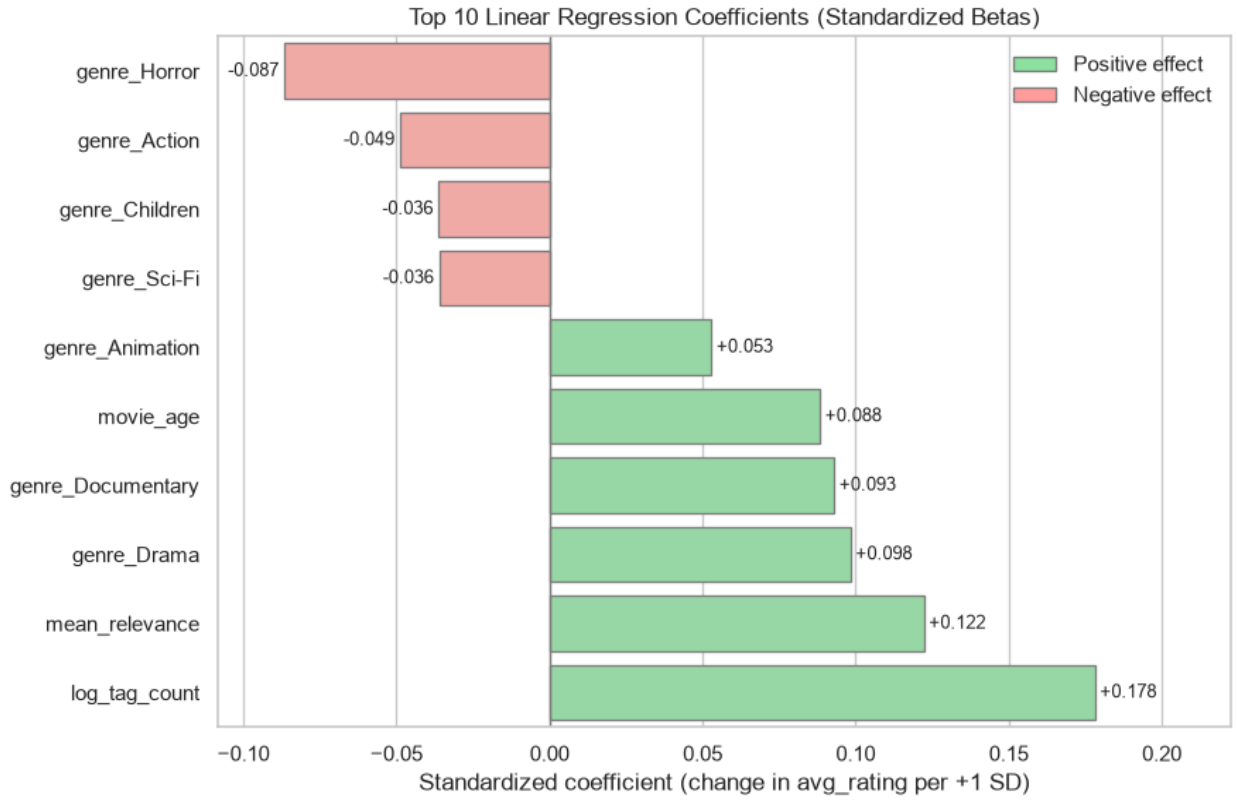


Figure 10: *Top 10 standardized coefficients of the Linear Regression model, ranked by absolute value. Green bars have positive effects and red bars have negative effects.*

- **Positive effects.** log_tag_count has the largest coefficient (about +0.18), followed by mean_relevance (+0.122), genre_Drama (+0.098), genre_Documentary (+0.093), movie_age (+0.088) and genre_Animation (+0.053). Movies that attract more tagging activity and have higher semantic relevance tend to receive higher ratings, and the positive movie_age coefficient

is consistent with the survivorship bias observed in Chart 2 (older movies that are still watched tend to be classics).

- **Negative effects.** genre_Horror (−0.087) has the strongest negative coefficient, followed by genre_Action (−0.049), genre_Children (−0.036) and genre_Sci-Fi (−0.036). The Horror result agrees with Chart 3, where Horror has the lowest mean rating.
- **Caution.** The coefficients describe associations, not causal effects. Genres overlap (one movie can carry several genre flags), so a coefficient measures a genre's effect given the movie's other genres, which is why Animation is positive while Children is negative. Linear Regression ranks log_tag_count first, whereas XGBoost ranks genre indicators first (Figure 9B): a signed linear coefficient and a tree-based importance score measure different things.

6.5. Cross-Validation

To verify stability, 5-Fold Cross-Validation (shuffled, random_state=42) was applied to the best model, XGBoost, on the training set.

Table 5: 5-Fold cross-validation results for XGBoost

Metric	**Mean ± Std**	**Fold 1**	**Fold 2**	**Fold 3**	**Fold 4**	**Fold 5**
RMSE	0.3533 ± 0.0061	0.3573	0.3527	0.3421	0.3546	0.3598
R^2	0.5065 ± 0.0077	0.5044	0.5039	0.5193	0.5090	0.4959

The cross-validated scores (RMSE = 0.3533, R^2 = 0.5065) are very close to the test-set scores (RMSE = 0.3524, R^2 = 0.5026), and the standard deviation across folds is small. This indicates that the model is stable and not significantly overfitted.

7. Conclusion

Key findings. This project predicted the average rating of 15,451 MovieLens movies from genre, movie age, community tagging and Tag Genome features (26 predictors). XGBoost performed best on the test set (MAE = 0.2572, RMSE = 0.3524, R^2 =

0.5026), reducing RMSE by about 29.5% relative to the mean baseline (0.4997), and its 5-Fold cross-validation results were stable (RMSE = 0.3533 ± 0.0061). Genre indicators (Documentary, Drama, Horror, Action) and the semantic relevance feature mean_relevance were the most influential predictors, consistent with the exploratory analysis in Section 4.

Practical implications. The model can estimate the expected reception of new or rarely rated movies from content and tagging metadata alone, which supports the cold-start problem in content-based recommendation systems and helps streaming platforms prioritize content.

Limitations.

- The model explains only about half of the variance in ratings ($R^2 \approx 0.50$) and regresses toward the mean at the extremes, overestimating poorly rated movies and underestimating the best-rated ones.
- The dataset contains no information on directors, cast, budget (IMDb/TMDb) or user demographics.
- Leakage-prone variables (rating_count, rating_std, rating_span_days) were excluded to keep the evaluation valid, which limits the attainable R^2.
- Only movies with at least 10 ratings were retained, so the results may not generalize to rarely rated movies.
- Genre indicators are not independent, because one movie can belong to several genres.
- Hyperparameters were set manually rather than tuned.

Future improvements.

1. Apply TF-IDF to tags_text to extract richer semantic features.
2. Join IMDb/TMDb metadata (director, cast, budget) through link.csv.
3. Tune hyperparameters with GridSearchCV or Optuna.
4. Test other gradient-boosting models such as LightGBM or CatBoost.